Innonsh CRM Suite - Master Security Implementation Report

Overview

This document serves as the formal record of the comprehensive Security Audit and Hardening process performed on the Innonsh CRM Suite. Over the course of 5 Sprints, the application was fortified against modern web vulnerabilities, bringing it closer to Enterprise-grade SOC2 and GDPR compliance standards.

🛡️ Sprint 1: Foundation & Boundary Security

What Was Done

1. **Centralized Environment Validation (`src/lib/env.js`)**: Integrated `zod` to validate all environment variables at startup. If critical secrets (like `JWT_SECRET`) are missing or too weak (less than 32 characters), the server will fail securely rather than running in a compromised state.
2. **Next.js Security Headers**: Enforced HTTP security headers (`Strict-Transport-Security`, `X-XSS-Protection`, `X-Content-Type-Options`, `X-Frame-Options: SAMEORIGIN`) globally via `next.config.js`.
3. **Edge-Compatible JWT Middleware (`src/middleware.js`)**: Replaced loose session checks with a robust edge middleware using the `jose` library. It validates the JWT for all `/api/*` routes and enforces Role-Based Access Control (RBAC).
4. **Strong Password Policies**: Updated password validation regex to strictly require 8+ characters, including uppercase, lowercase, numbers, and special symbols across both API endpoints and the frontend UI.
5. **Dependency Vulnerability Remediation**: Resolved moderate vulnerabilities by running `npm audit fix`, updating packages like `postcss`.

Testing Steps

- **Environment Validation**: Temporarily remove `JWT_SECRET` from your `.env` file and attempt to start the server (`npm run dev`). The server should crash with a Zod validation error.
- **Security Headers**: Open browser developer tools (Network tab), make a request to the server, and verify the presence of `X-Frame-Options` and `Strict-Transport-Security` headers on the response.
- **Middleware & RBAC**: Attempt to access `/api/users` via an API client (like Postman) without a Bearer token or cookie. Verify you receive a `401 Unauthorized` error.

- **Password Policies**: Attempt to register a new user with the password `password123`. Verify the API and frontend reject it, demanding higher complexity.

🔒 Sprint 2: API Security & Token Architecture

What Was Done

1. **Strict Payload Validation (Zod)**: Applied `zod` schema parsing to all authentication and user endpoints (`login`, `register`, `users`, `forgot-password`, `reset-password`, `auth/me`).
2. **API Rate Limiting**: Added a sliding-window Map rate limiter to `middleware.js`. Auth routes are restricted to **5 requests/minute**, while general API routes are restricted to **60 requests/minute**.
3. **Refresh Token Architecture**: Reduced the main JWT access token lifespan from 7 days to **15 minutes**. Introduced a 7-day `refresh_token` stored in an HTTP-Only cookie, scoped strictly to the `/api/auth/refresh` endpoint.
4. **Input Sanitization**: Utilized Zod's `.trim()` and `.toLowerCase()` sanitization methods on incoming strings to mitigate injection vectors.

Testing Steps

- **Payload Validation**: Send a POST request to `/api/auth/login` with an empty object `{}`. Verify you receive a `400 Bad Request` with a specific Zod error message (e.g., "Invalid email address syntax").
- **Rate Limiting**: Refresh the login page rapidly or send 6 consecutive POST requests to `/api/auth/login` via Postman within 60 seconds. The 6th request should return a `429 Too Many Requests` error.
- **Refresh Tokens**:
    1. Log into the application and inspect your browser cookies. You should see two cookies: `token` (15m expiry) and `refresh_token` (7d expiry).
    2. Wait 15 minutes (or manually delete the `token` cookie) and trigger a request to `/api/auth/refresh`. Verify a new `token` cookie is issued.

📊 Sprint 3: Monitoring, Audit & Compliance

What Was Done

1. **Winston Logger**: Implemented professional structured JSON logging via `winston` in `src/lib/logger.js`.
2. **Audit Trails**: Injected `auditLog()` hooks into critical lifecycle events:

    - `USER_LOGIN_SUCCESS` / `USER_LOGIN_ERROR` (capturing IPs).
    - `USER_REGISTER_REQUEST`.
    - `USER_CREATED` / `USER_CREATE_ERROR`.
3. **GDPR / SOC2 Data Endpoints**:
    - Created `/api/users/[id]/export` allowing users to download their personal data footprint.
    - Created `/api/users/[id]/delete` allowing the CRM Owner to securely and compliantly purge user records.

Testing Steps

- **Audit Logs**: Perform a login, then check the server console (terminal running `npm run dev`). You should see a formatted Winston JSON log indicating `AUDIT: USER_LOGIN_SUCCESS` with the user ID and IP address.
- **Data Export**: Send a GET request to `/api/users/[your_user_id]/export` using your valid session cookie. Verify the response is a clean JSON object containing your data without sensitive fields (like passwords).
- **Data Deletion Privilege**: Log in as a `sales_rep` and attempt to send a DELETE request to `/api/users/[some_id]/delete`. Verify it is rejected with a `403 Forbidden`.

🏗️ Sprint 4: Infrastructure & CI/CD Security

What Was Done

1. **Dependabot**: Created `.github/dependabot.yml` to run weekly automated dependency vulnerability scans for `npm`.
2. **CodeQL SAST Scanning**: Created `.github/workflows/codeql.yml` to automatically perform deep Static Application Security Testing on pull requests and pushes to the main branch.
3. **Disaster Recovery**: Authored `docs/BACKUP_AND_DR.md` outlining specific fallback procedures for Supabase Database failures and Vercel hosting outages.

Testing Steps

- **CI/CD Actions**: Navigate to the "Actions" tab in your GitHub repository. Verify that the "CodeQL Security Scan" workflow exists and completes successfully against the `main` branch.
- **Dependabot**: Navigate to the "Security" tab in GitHub -> "Dependabot" and verify it is active and tracking the `package.json`.

□ Sprint 5: Enterprise Security Features

What Was Done

1. **Database Tracking**: Updated `src/lib/models/User.js` to natively support `mfaEnabled` flags and an `activeSessions` array.
2. **Active Session Harvester**: Upgraded `/api/auth/login` to automatically extract the user's IP address and `User-Agent` browser string, pushing it to the `activeSessions` array upon successful login.
3. **Security Dashboard UI**: Built the UI framework for the Enterprise Security Settings page at `src/app/dashboard/security/page.js`, providing toggles for 2FA and options to terminate active sessions.

Testing Steps

- **Session Tracking**: Log into the application via your browser. Open your MongoDB GUI (or Supabase Dashboard), locate your User record, and verify the `activeSessions` array contains a new entry with your current IP address, token, and User-Agent string.
- **Security UI**: Navigate to `http://localhost:3000/dashboard/security` in the browser. Verify the new Enterprise Security interface loads, displaying the MFA toggle and the Active Sessions list framework.